Ch. 7 Notes
Heaps and Priority Queues

What a heap is

A **complete** binary tree with the heap property:

- **Max-heap**: every parent $\geq$ both children. Root is max.
- **Min-heap**: every parent $\leq$ both children. Root is min.

Not a BST: left and right siblings are unordered w.r.t. each other. No in-order traversal, no search, no range query.

Array representation

Zero-indexed, root at 0. For node at index i:

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \ \text{left}(i) = 2i+1, \ \text{right}(i) = 2i+2$$

Works only because the tree is complete (no gaps).

Why array beats pointers

Pointer tree	Array heap
3 ptrs / node	no per-node overhead
cache miss per child hop	children 1–2 cells ahead
`new`/`delete`	`vector` amort. $O(1)$

Percolate up (insert fixup)

```
while (i > 0) {
    int p = (i - 1) / 2;
    if (h[i] <= h[p]) break;
    std::swap(h[i], h[p]);
    i = p;
}
```

Percolate down (extract fixup)

```
while (2*i + 1 < n) {
    int L = 2*i + 1, R = 2*i + 2, big =
        L;
    if (R < n && h[R] > h[L]) big = R;
    if (h[i] >= h[big]) break;
    std::swap(h[i], h[big]);
    i = big;
}
```

Watch: swap with the *larger* child, not an arbitrary child.

Insert / extract

- **Insert**: push_back, then percolate up.
- **Extract**: save root, move last to root, pop_back, percolate down.

Both $O(\log n)$; peek $O(1)$.

Cost table

	peek	pop	push
sorted array	$O(1)$	$O(1)$	$O(n)$
unsorted array	$O(n)$	$O(n)$	$O(1)$
balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$
heap	$O(1)$	$O(\log n)$	$O(\log n)$

Heapify (bottom-up build)

For i from $\lfloor n/2 \rfloor - 1$ down to 0, call `percolateDown(i)`. Total $O(n)$, not $O(n \log n)$: most nodes are near the bottom and do almost no work ($\sum n/2^{i+1} \cdot i = O(n)$).

Heapsort

1. Heapify in place – $O(n)$.
2. For `end = n-1` down to `1`: swap `a[0]` with `a[end]`, then `percolateDown(0, size=end)`. – $O(n \log n)$.

In place, $O(1)$ extra, **not stable**. Worst case $O(n \log n)$ – why introsort falls back to it.

Priority queue ADT

Operations: `push(x)`, `pop()` (returns extremum), `peek()`, `size()`, `empty()`. Usually implemented as a heap. Uses: Dijkstra / A* / Prim, event simulation, top-k, process scheduling.

`std::priority_queue`

```
std::priority_queue<int> maxpq;
                // max-heap
std::priority_queue<int, std::vector<int
    >,
    std::greater<int>> minpq;
                    // min-heap
```

`.top()` returns **const ref** – no in-place priority change. To update priority: pop + push, or use indexed heap.

Treap (sneak preview of balance)

Each node carries (key, random priority). BST order on keys, heap order on priorities. Insert as BST leaf, then rotate up while priority > parent's. Rotations preserve BST order. Expected $O(\log n)$ all ops; worst-case $O(n)$ (unlucky priorities).

Top gotchas

- Index arithmetic off-by-one: `(i-1)/2`, not `i/2`. Must use int division.
- Swapping with the smaller child in `percolateDown` – creates a new violation below.
- Using a heap where you need ordered traversal or search (use a BST / `std::map`).
- Treating `pop()` on empty as safe – UB in C++.